UNIVERSITÉ
CAEN
NORMANDIE

Projet 2 : Analysis of Sorting Algorithms

Mehmet Tuna Ozkalkanli
Andrea Gjoreska
Mila Bucevska

2026

Central Question

Question : How does the efficiency of each algorithm, based on different criteria, such as comparisons, swaps, data accesses, and execution time, vary depending on the quantity and distribution of disorder in the dataset input?

- Create a data generator with different disorder levels
- Implement the sorting algorithms with different behaviors
- Build a real-time visualization interface
- Track comparisons, swaps, and data accesses
- Analyze performance through benchmarking and compare the results

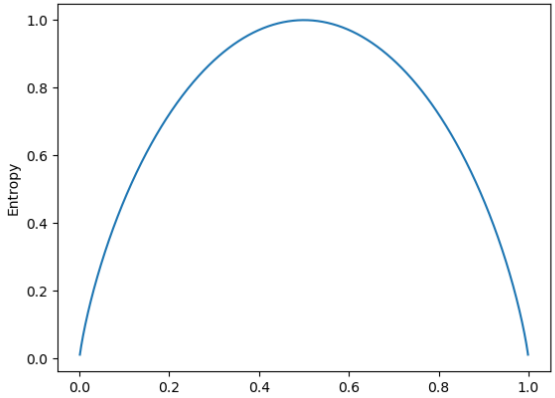
- **Mehmet Tuna Ozkalkanli**
  - Data generators
  - QuickSort, BogoSort
  - Benchmark and CSV export
  - Analysis
- **Andrea Gjoreska**
  - Global structure and MVC architecture
  - Visualization and compare mode
  - InsertionSort, MergeSort, CocktailShakerSort, PancakeSort
- **Mila Bucevska**
  - Event tracking and interface
  - BubbleSort, BucketSort, CombSort
  - Statistics window and graphs

Definition: Entropy measures how unpredictable the list structure is based on the proportion of displaced elements.

- Generates lists with a **target entropy level**
- Starts from a **sorted list**
- Introduces controlled **displacement**
- Keeps a valid **permutation**

Idea: Control *how disordered* a list is

$$H(p) = H(1-p) = -p\log_2(p) - (1-p)\log_2(1-p)$$



1. Choosing the number of displaced elements

$$H(p) = -p \log_2(p) - (1-p) \log_2(1-p)$$

- $p = \frac{d}{n}$
- Try $d \in [0, n/2]$
- Choose d minimizing:

$$|H(p) - \text{target}|$$

Symmetry: $H(p) = H(1-p)$

- If target > 0.5, return $n - d$ to exploit symmetry and avoid generating identical lists

2. Applying the displacement

- Select d random indices
- Apply a cyclic permutation

Example:

$$[0, 1, 2, 3, 4, 5], \quad \text{indices} = [4, 1, 5]$$

$$4 \to 1, \quad 1 \to 5, \quad 5 \to 4$$

- Target entropy must be in $[0, 1]$
- Skip $d = 1$ (impossible case)
- Store:
  - `lastDisplaced`
  - `lastEntropy`
- Actual entropy $\neq$ target (approximation)

Key takeaway:
Generating data with **controlled "randomness"**

- **Algorithms tested:** BubbleSort, CocktailShakerSort, InsertionSort, MergeSort, QuickSort, CombSort, PancakeSort
- **Input sizes:** $n \in \{100, 500, 1000\}$ - each configuration repeated 4 times to reduce randomness
- **Generator types:**
  - Random
  - Entropy
  - Reverse entropy
- **Metrics collected:** number of comparisons, swaps, memory accesses, execution time

Algorithm	**Comparisons (n=100)**	**(n=1000)**	**Growth**	**Complexity**
MergeSort	540	8.700	×16	O(n log n)
QuickSort	650	11.000	×17	O(n log n)
CombSort	1.300	23.000	×18	O(n log n)
InsertionSort	2.500	250.000	×100	$O(n^2)$
CocktailShaker	3.900	380.000	×97	$O(n^2)$
BubbleSort	4.800	498.000	×104	$O(n^2)$
PancakeSort	5.148	501.498	×97	$O(n^2)$

Algorithm	**Comparisons (n=100)**	**(n=1000)**	**Growth**	**Complexity**
MergeSort	356	5.044	×14	O(n log n)
QuickSort	4950	499.500	×100	O(n^2) – worst case
CombSort	1.003	18.713	×18	O(n log n)
InsertionSort	99	999	×10	O(n) – best case
CocktailShaker	99	999	×10	O(n) – best case
BubbleSort	99	999	×10	O(n) – best case
PancakeSort	5.148	501.498	97	O(n^2) – not adaptive

Algorithm	**Comparisons (n=100)**	**(n=1000)**	**Growth**	**Complexity**
MergeSort	424	6.950	×16	O(n log n)
QuickSort	2.900	48.650	×16	O(n log n)
CombSort	1.201	21.700	×18	O(n log n)
InsertionSort	195	15.100	×77	$O(n^2)$
CocktailShaker	391	22.750	×58	$O(n^2)$
BubbleSort	2.750	118.000	×42	$O(n^2)$
PancakeSort	5.148	501,498	×97	$O(n^2)$

Algorithm	**Comparisons (n=100)**	**(n=1000)**	**Growth**	**Complexity**
MergeSort	446	7.700	×17	O(n log n)
QuickSort	1.650	33.100	×20	O(n log n)
CombSort	1.201	22.700	×19	O(n log n)
InsertionSort	521	37.000	×71	$O(n^2)$
CocktailShaker	767	60.500	×79	$O(n^2)$
BubbleSort	4.640	494.000	×106	$O(n^2)$
PancakeSort	5.148	501.498	×97	$O(n^2)$

Algorithm	**Comparisons (n=100)**	**(n=1000)**	**Growth**	**Complexity**
MergeSort	560	8.694	×16	O(n log n)
QuickSort	662	11.450	×17	O(n log n)
CombSort	1.250	22.700	×18	O(n log n)
InsertionSort	2.300	216.800	×94	$O(n^2)$
CocktailShaker	3.250	320.000	×100	$O(n^2)$
BubbleSort	4.850	499.000	×100	$O(n^2)$
PancakeSort	5.148	501.498	×97	$O(n^2)$

Algorithm	**Comparisons (n=100)**	**(n=1000)**	**Growth**	**Complexity**
MergeSort	316	4.932	×16	O(n log n)
CombSort	1.102	19.712	×18	O(n log n)
QuickSort	4.950	499.500	×101	$O(n^2)$
InsertionSort	4.950	499.500	×101	$O(n^2)$
BubbleSort	4.950	499.500	×101	$O(n^2)$
CocktailShaker	5.000	500.000	×100	$O(n^2)$
PancakeSort	5.148	501.498	×97	$O(n^2)$

Algorithm	**Comparisons (n=100)**	**(n=1000)**	**Growth**	**Complexity**
MergeSort	385	6.950	×18	O(n log n)
CombSort	1.150	21.700	×19	O(n log n)
QuickSort	2.500	53.000	×21	O(n log n)
InsertionSort	4.890	486.800	×100	$O(n^2)$
BubbleSort	4.950	499.500	×101	$O(n^2)$
CocktailShaker	4.995	499.750	×100	$O(n^2)$
PancakeSort	5.148	501.498	×97	$O(n^2)$

Algorithm	**Comparisons (n=100)**	**(n=1000)**	**Growth**	**Complexity**
MergeSort	430	7.700	×18	O(n log n)
CombSort	1.200	22.700	×19	O(n log n)
QuickSort	2.200	23.000	×10	O(n log n)
InsertionSort	4.675	466.000	×100	$O(n^2)$
BubbleSort	4.950	499.500	×101	$O(n^2)$
CocktailShaker	4.980	499.000	×100	$O(n^2)$
PancakeSort	5.148	501.498	×97	$O(n^2)$

Algorithm	**Comparisons (n=100)**	**(n=1000)**	**Growth**	**Complexity**
MergeSort	534	8.700	×16	O(n log n)
QuickSort	650	12.000	×18	O(n log n)
CombSort	1.250	22.700	×18	O(n log n)
InsertionSort	3.090	290.000	×94	$O(n^2)$
CocktailShaker	4.150	430.000	×104	$O(n^2)$
BubbleSort	4.910	499.500	×102	$O(n^2)$
PancakeSort	5.148	501.498	×97	$O(n^2)$

- The experiments confirm that **input structure strongly influences performance**
- Three behaviors emerge:
  - **Strongly structure-sensitive:** BubbleSort, CocktailShakerSort, InsertionSort
  - **Mostly stable across inputs:** MergeSort
  - **Intermediate / partially sensitive:** CombSort, QuickSort
- Adaptive algorithms perform extremely well on low entropy input, but degrade quickly toward $O(n^2)$ as disorder increases
- In contrast, $O(n \log n)$ algorithms remain **stable and predictable** regardless of input distribution

- **MergeSort** is the most consistent, fast and reliable algorithm
  - Always close to $O(n \log n)$
  - Unaffected by entropy or input order
- **QuickSort** can be the fastest (beating MergeSort with 1-2 ms in some cases) in practice on random/high-entropy data
  - Efficient on average
  - But unstable due to pivot choice (worst-case degradation)
- **CombSort** is a clear improvement over BubbleSort
  - Same idea but with gaps
  - Much fewer comparisons in practice
  - Scales closer to $O(n \log n)$ behavior **experimentally**

- **Nearly sorted data:**
  - Best choice: **InsertionSort**
- **Random or high entropy data:**
  - Best choice for speed in lower list sizes: **QuickSort**
  - Best choice for overall stability: **MergeSort**
- **Unknown input:**
  - Best choice: **MergeSort**
- **Simple improvement over BubbleSort:**
  - Use **CombSort**

Overall Winner

MergeSort is the overall winner.

- It consistently performs **50x fewer comparisons** than quadratic algorithms on large inputs (e.g., ∼8,700 vs ∼500,000 at $n = 1000$)
- Execution time remains extremely lowacross all configurations
- Unlike QuickSort, it does not suffer from worst-case degradation
- Therefore, it is the most predictable and reliable algorithm overall